Checking learned dissipativity conditions over the whole region: all-VSG Kundur (first pass)

Status: a reproduction with a measured gap. It is not a statement about the paper's own controller. The paper (Wang, Wang, Zhuang, Moffat, Dörfler, arXiv:2510.22324) publishes no code, and several training hyperparameters are not given. Everything below is on my rebuilt example, so the gap could come from my reproduction rather than from the method.

Setup

- **System:** Kundur two-area network with all four generators as VSGs.
- VSG parameters as in the paper: virtual impedance j0.3, H = 4 s, damping 5 pu on the 900 MVA base.
- Second-order swing model, Kron-reduced, 0.5 ms ZOH step, post-fault topology with one B8-B9 circuit open.
- Without control, the system loses synchronism after the fault, matching the paper's Fig. 6.
- **Controller:** the paper's parametrization and losses.
- Matrix networks $W(x)$, $Q(x)$, $S(x)$, $R^{-1}(x)$, with $\pi(x) = -R^{-1}(x)S(x)^{T}x$.
- Dissipativity loss (eq. 12), stability-condition loss on $\Phi = S\,R^{-1}\,S^{T} - Q$ (eq. 13), cost shaping and L1.
- Data: 2,500 one-second trajectories with uniformly random u, giving 98,069 in-region tuples.
- Deviations, all stated in `results/train_config.json`:
    - an input-range penalty so $\pi(x)$ stays inside the trained u-range (Theorem 2's hypothesis)
    - mean-plus-max aggregation instead of a pure batch max
    - my own choices for the epsilons and for the cost-shaping form
- **Model fixes made along the way:**
- the post-fault equilibrium solve diverged, and is now a slack-bus solve with continuation
- speed was double-counted in the step function
- inertia is now in per-unit speed units (M = 2H·S/S_base)

What was checked

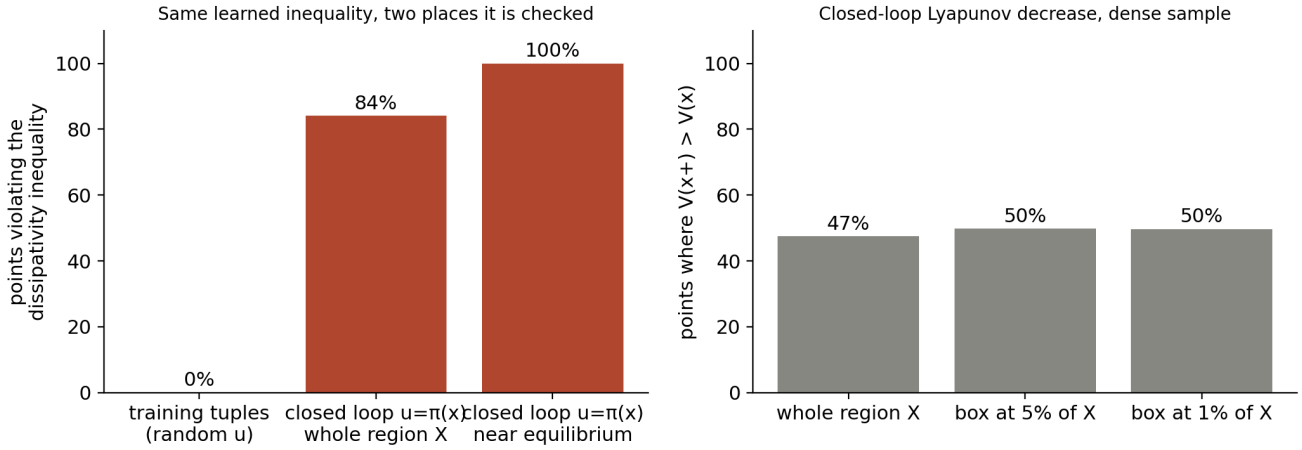
For the dissipativity inequality (5b), the closed-loop Lyapunov decrease $V(x^{+}) \le V(x)$, the stability condition $\Phi(x) \succ 0$, and the input range: 1. **On the training tuples**, with the random u the paper trains on. 2. **Along the closed loop**, with $u = \pi(x)$. This is the case Theorem 2's stability argument actually consumes, and $u = \pi(x)$ is the minimizer of the supply rate $s(x, u)$. Checked by a dense uniform sample (200,000 points over the paper's region X: $|\Delta\delta| \le 2\pi$, $|\Delta\omega| \le 0.1$, and 50,000 in boxes at 5% and 1% of X) and by multi-start gradient attacks (512 starts).

Result

Check	Training tuples (random u)	Closed loop, whole X	Closed loop, box at 1% of X
Dissipativity inequality violated	**0%**	**84%**	**100%**
Lyapunov decrease violated	n/a	47%	50%

Check	Training tuples (random u)	Closed loop, whole X	Closed loop, box at 1% of X
$\Phi(x) \succ 0$ violated	n/a	23%	0%

- **The inequality holds at every training point and fails almost everywhere along the closed loop.**
- The reason is mechanical. With random u, the supply rate is large (median 85 against a storage change of about 0.006 per step), so the inequality is easy to satisfy.
- At $u = \pi(x)$, the supply rate becomes $-x^T\Phi x$, which is small, and that is exactly where the inequality has to hold for the stability proof.
- Random excitation almost never samples that minimizing input.
- **Consistent with that, the closed loop does not stabilize the fault case** in my reproduction. The trajectory diverges.
- **$\Phi \succ 0$ was only marginally learned.** Its smallest eigenvalue sits near zero, because the softplus loss saturates at this scale. That is a training limitation of my run.



What I could not do yet

- **Sound certificates (CROWN branch-and-bound) on this architecture.** auto_LiRPA 0.7.2 does not support GELU or softplus, both of which the paper's networks use. Interval arithmetic is sound but too loose for $V(x) - V(x^+)$, a difference of nearly equal values. Only the search tiers (dense sampling and attacks) are reported, and they are searches, not certificates.
- **Current limiting:** not run.

Proposed next step

Enforce the inequality where the proof needs it. Either add the closed-loop condition at $u = \pi(x)$ to training, or sample u adversarially, toward the supply-rate minimizer, instead of uniformly. Then repeat the region check. With ReLU or other verifier-supported activations, the same check can be made a sound CROWN certificate. If the authors' trained controller can be shared, the same pipeline can check it directly, which would separate a reproduction issue from a property of the method.

Reproduce

```
cd code
python train.py 2500 60     # ~17 min CPU (dataset cached after first run)
python verify.py 200000 512 # ~2 min
python fig.py
```